

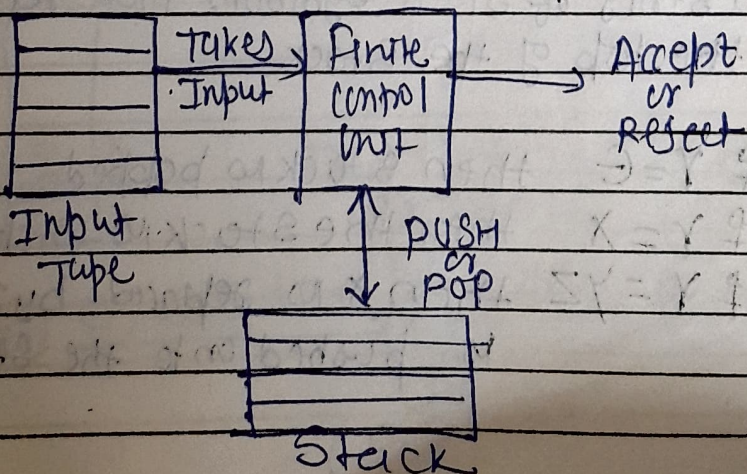
Push down Automata

A pushdown Automata (PDA) is a way to implement a Context Free Grammar in a similar way we design finite Automata for Regular Grammar.

- It's more powerful than FSM
- FSM has very limited memory but PDA has more memory.
- PDA = finite State Machine + A stack
- A Stack is a way we arrange elements one on top of another.
- A Stack does two basic operations:
 PUSH: A new element is added at the top of the stack
 POP: The top element of the stack is read and removed.

A pushdown Automata has 3 components.

- 1) An Input Tape
- 2) A finite control unit
- 3) A stack with infinite size.



$$\delta = Q \times \Sigma \cup \{\epsilon\} \times \Gamma \rightarrow Q \times \Gamma^*$$

Date / /

Page No.:

PDA formal Definition.

A Pushdown Automata is formally defined by 7 Tuples as below

$$P = (Q, \Sigma, \Gamma, \delta, q_0, z_0, F)$$

$Q \Rightarrow$ A finite set of States

$\Sigma \Rightarrow$ A finite set of Input Symbols

$\Gamma \Rightarrow$ A finite stack alphabet

$\delta \Rightarrow$ The transition function.

$q_0 \Rightarrow$ The start state.

$z_0 \Rightarrow$ The start stack symbol

$F \Rightarrow$ The set of final / Accepting states.

δ takes as argument a triple $\delta(q, ax)$ where

i) q is a state in Q

ii) a is either an input symbol in Σ or $a = \epsilon$

iii) x is a stack symbol, that is a member of Γ

The output of δ is finite set of pairs (p, γ) where:

Q p is a new state

γ is a string of stack symbols that replaces x at the top of the stack

Ex: If $\gamma = \epsilon$ then stack is popped

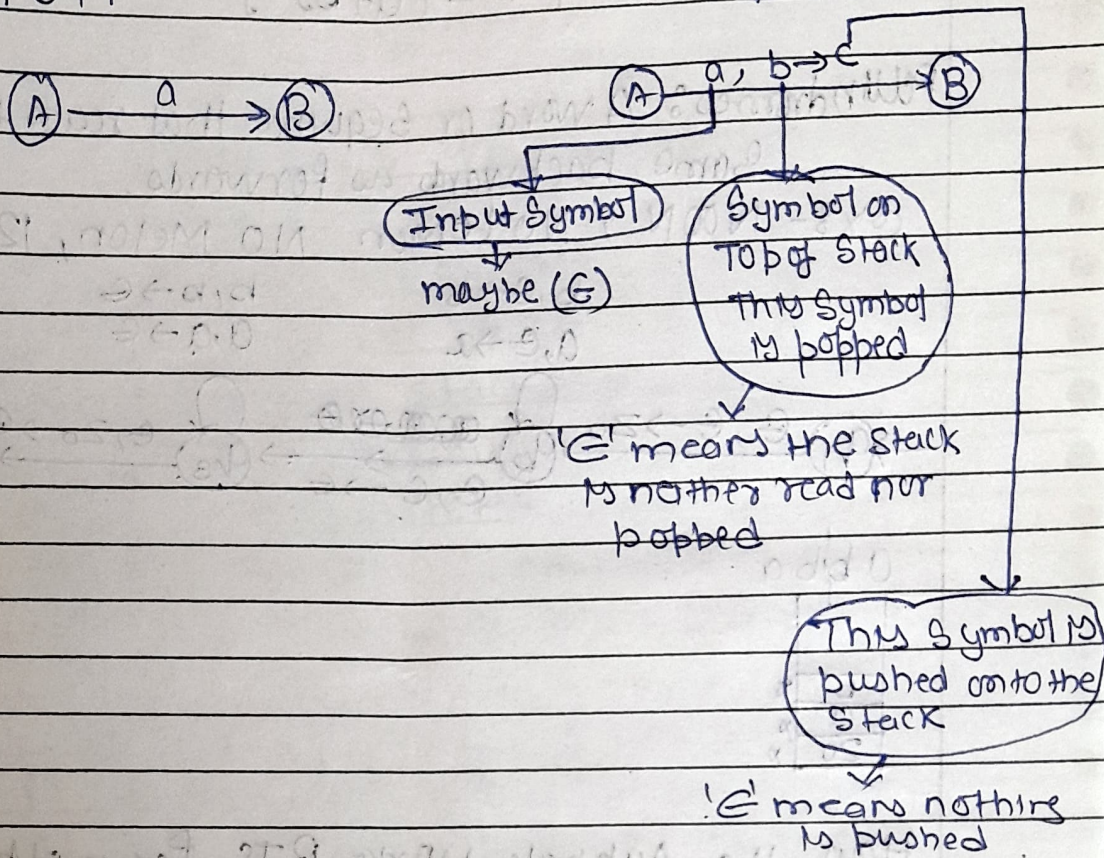
If $\gamma = x$ then the stack is unchanged

If $\gamma = yz$ then x is replaced by z and y is pushed onto the stack

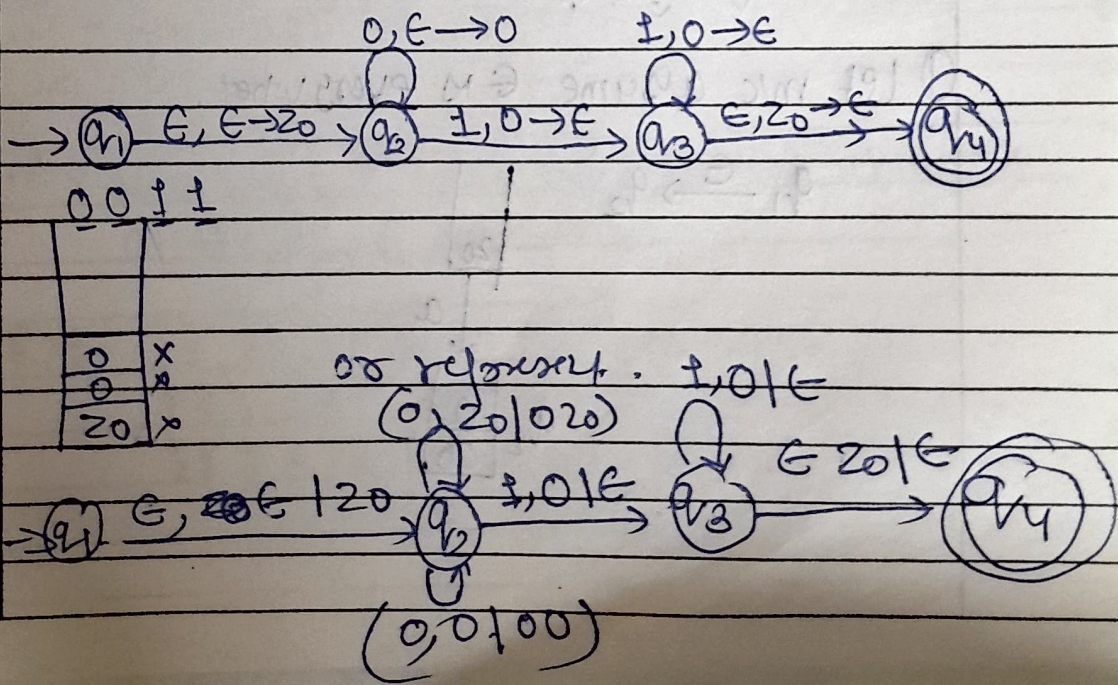
Pushdown Automata (Graphical Notation)

FSM

PDA



Q Construct PDA that accepts $L = \{0^n 1^n \mid n \geq 0\}$
NPDA



Construct PPA that accepts even palindromes of the form.

$$L = \{ w w^R \mid w = (a+b)^+ \}$$

Palindromes: A word or sequence that reads the same backwards as forwards.

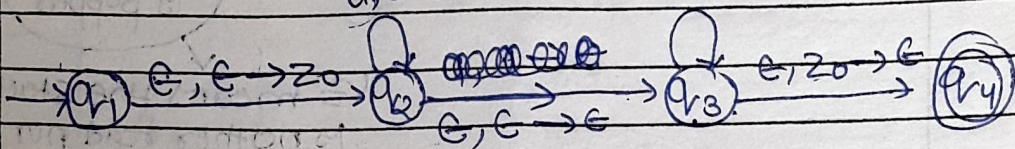
Ex: - NOON, NO LEMON, NO Melon, 123321

$$b, \epsilon \rightarrow b$$

$$b, b \rightarrow \epsilon$$

$$a, \epsilon \rightarrow a$$

$$a, a \rightarrow \epsilon$$



abba

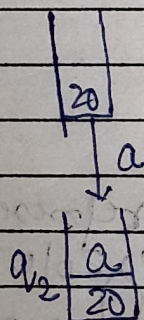
b	x
a	x
z0	x

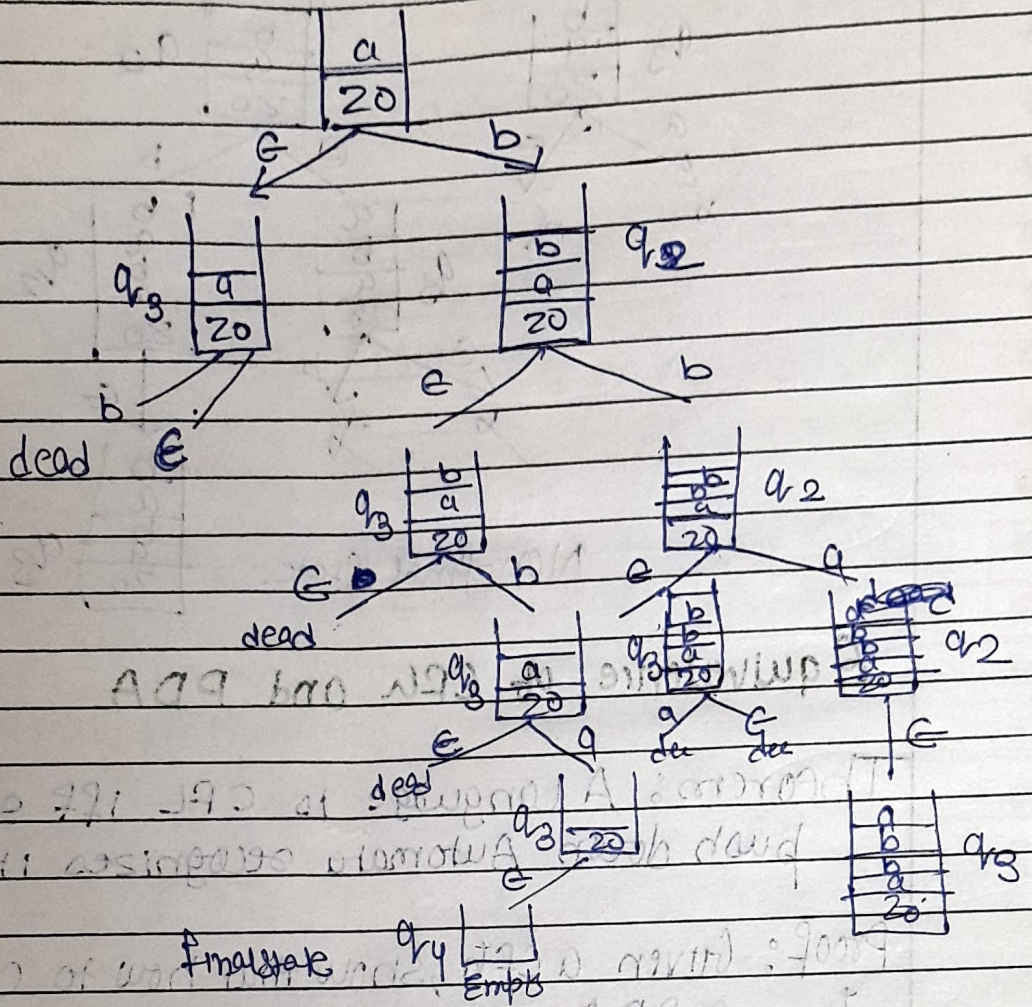
How the Automata works BTS. For midpoint of string.

$$\epsilon x : \epsilon a \epsilon b \epsilon b \epsilon a \epsilon$$

① Let me assume ϵ M everywhere.

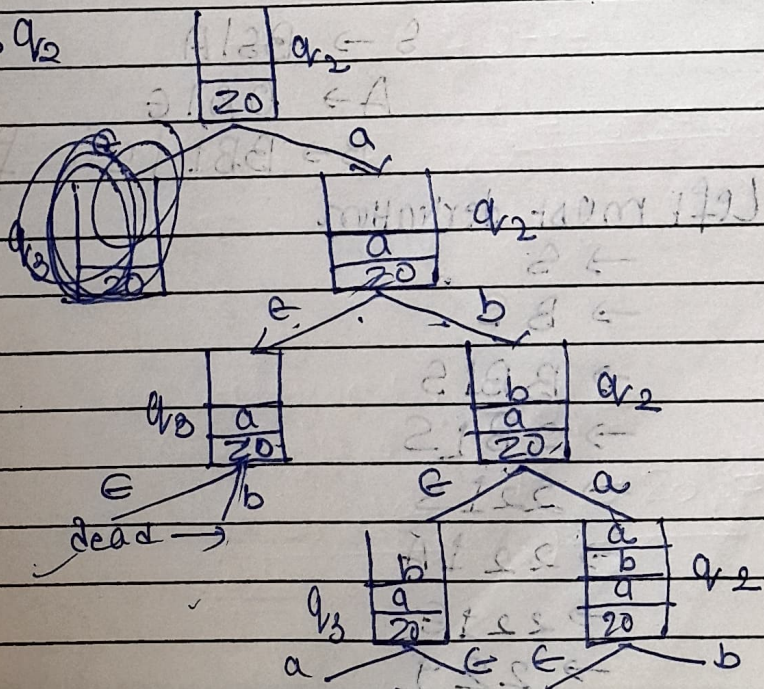
$$q_1 \xrightarrow{\epsilon} q_2$$

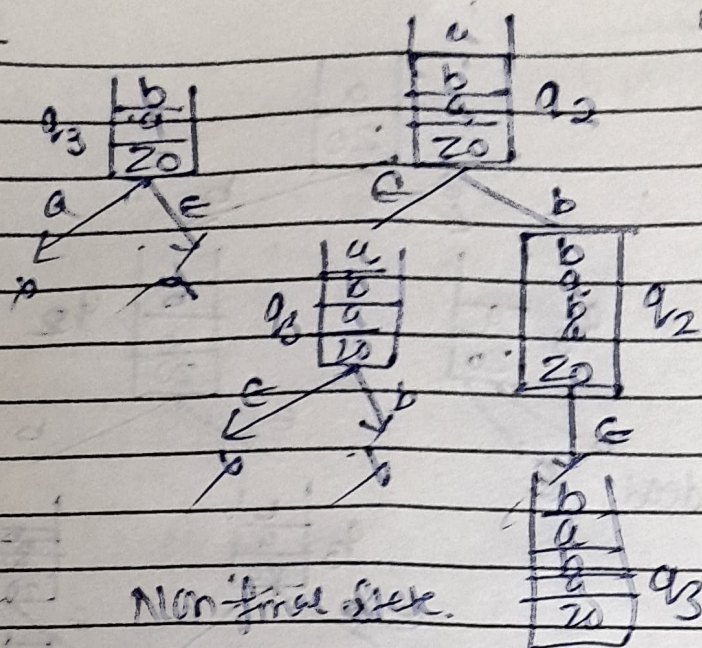




$abab \rightarrow \epsilon a \epsilon b \epsilon a \epsilon b \epsilon$

$q_1 \xrightarrow{\epsilon} q_2$





Equivalence of CFL and PDA

Theorem: A language is CFL iff some push down Automata recognizes it

Proof: Given a CFL, show that how to construct a PDA that recognizes it.

Given Grammar —

$$S \rightarrow BSIA$$

$$A \rightarrow OAIG$$

$$B \rightarrow BB112$$

Find or Build a PDA

Left most derivation

$\rightarrow S$

$\rightarrow BS$

$\rightarrow BBIS$

$\rightarrow 2BIS$

$\rightarrow 22IS$

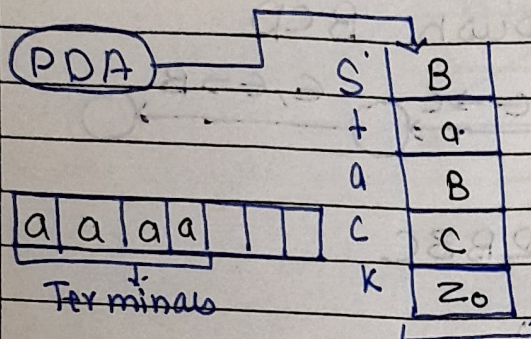
$\rightarrow 22IA$

$\rightarrow 221E$

$\rightarrow 221$

General Form

aaaa BaBc
 Terminals Rest



Left Most Derivation: $S \rightarrow \dots aaaa \boxed{BaBc}$

At each step expand left most Derivation

E.g.

Prod. Rule: $B \rightarrow ASAXBA$

Pop B from stack and push the production

Steps:-

① Match stack Top to a Rule

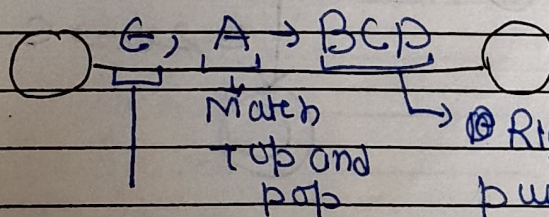
② pop Stack

③ push R.H.S of Rule onto stack

$\rightarrow \dots aaaa \boxed{ASAXBAaBc}$

Rule

$A \rightarrow BCD$

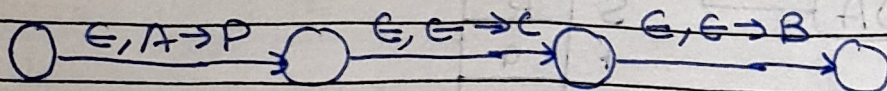


I/p not advanced

∴ We can't push more than 1 element to the stack at one time

4	3	b
c	a	2
		0

In order ~~to~~ push BCD



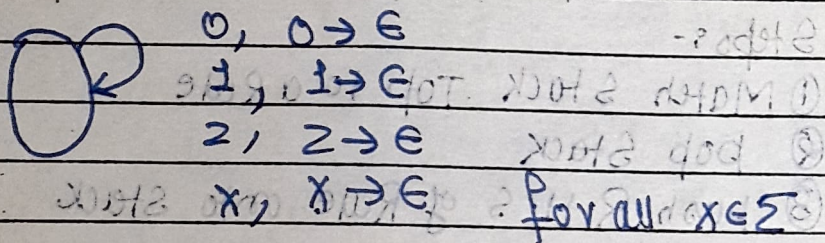
Rule: $A \rightarrow 0102B3C$

PDA

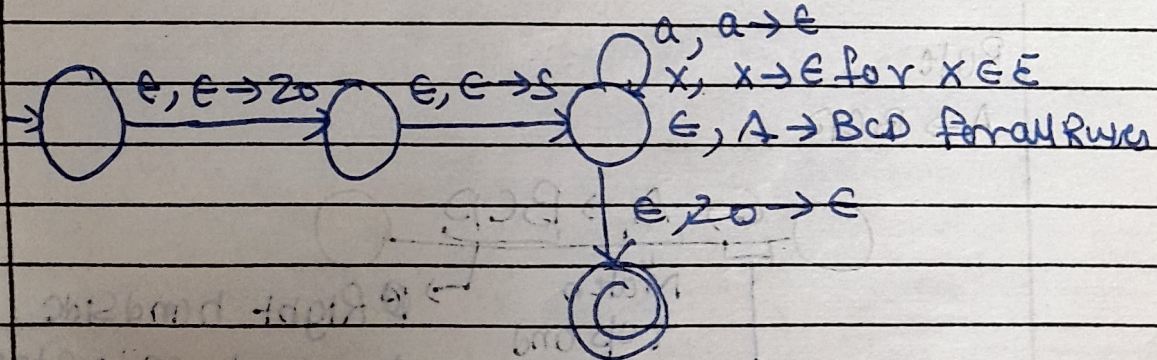
... 4 0 1 0 2 ...

0 1 0 2 B 3 C ... z₀

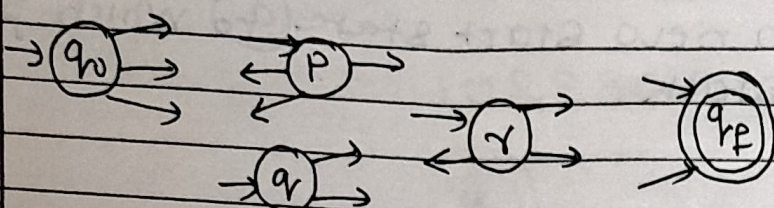
Match Terminal Symbol to the Stack Top
and pop them and advance it



The final PDA



Proof :- Given a PDA, show how to construct a CFL that recognizes the same language



S1: Simplify PDA

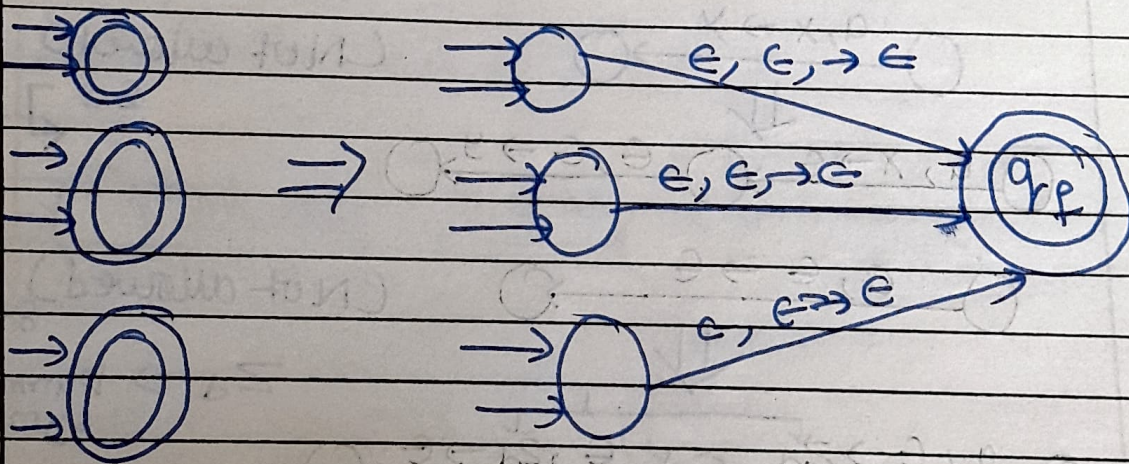
S2: Build the CFL

There will be a Non-Terminal for every pair of states: $A_{pq}, A_{qr}, A_{rq}, \dots$

The Starting Non-Terminal will be: $A_{q_0 q_f}$

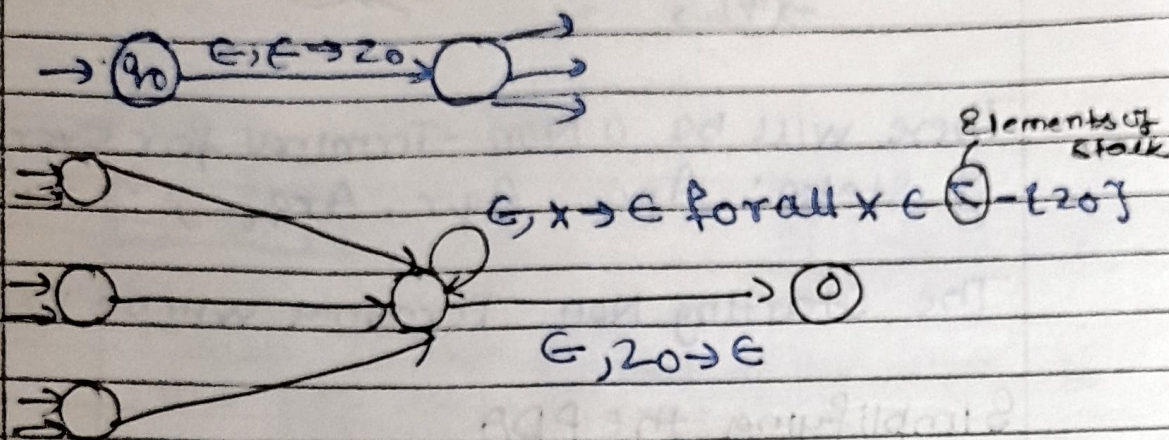
Simplifying the PDA

I) The PDA should have only one final accept state

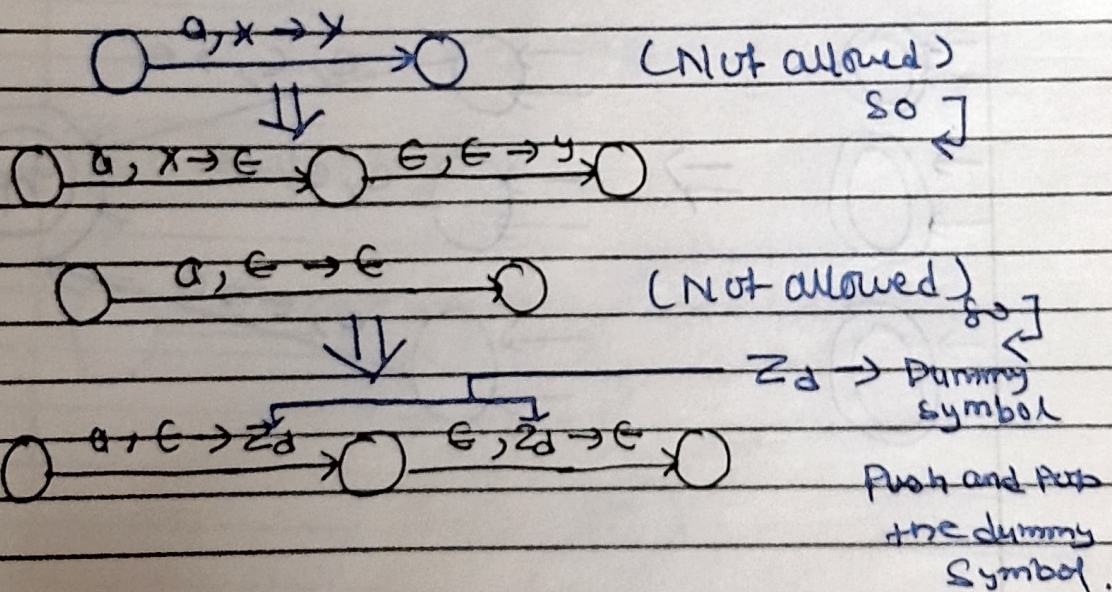


2) The PDA should empty its stack before accepting.

→ create a new start state (q_0) which pushes (z_0) to the stack.



3) Make sure each transition either Pushes or Pops but does not do both.



→ Start with an empty stack and finish with an empty stack.

Don't let stack underflow / overflow..

Main IDEA.

Consider two states P and q in the PDA
Could we go from p to q without stack underflow
and maintaining an empty stack at the beginning
and end? OR if something were already on the
stack they would never be touched.

What strings would do that?

We will create a Non Terminal A_{pq} in our
grammar.

A_{pq} will generate exactly those strings that
will take us from p to q maintaining all the
above stack conditions.

DPDA pushdown
(Deterministic Pushdown Automata)

NPDA
(Non-Deterministic Pushdown Automata)

1. It is less powerful than NPDA

It is more powerful than DPDA.

Ex:- We can only construct DPDA for odd-length palindrome & not for even length palindrome.

Ex:- NPDA can be constructed for both even-length & odd length palindromes.

2. It is possible to convert every DPDA to a complementing NPDA

It is not possible to convert every NPDA to a complementing DPDA.

3. There is only one state transition from one state to another state for an input symbol.

There may or may not be more than one state transition from one state to another state for ~~some~~ same input symbol.

4. Centre symbol is known
Ex:- W C W R

Centre symbol is not known
Ex:- W W R

One Stack PDA

$$\delta = Q \times E \cup \{ \epsilon \} \times \Gamma \times \Pi \rightarrow Q \times \Pi^*$$

Date ___/___/___

Page No.: ___

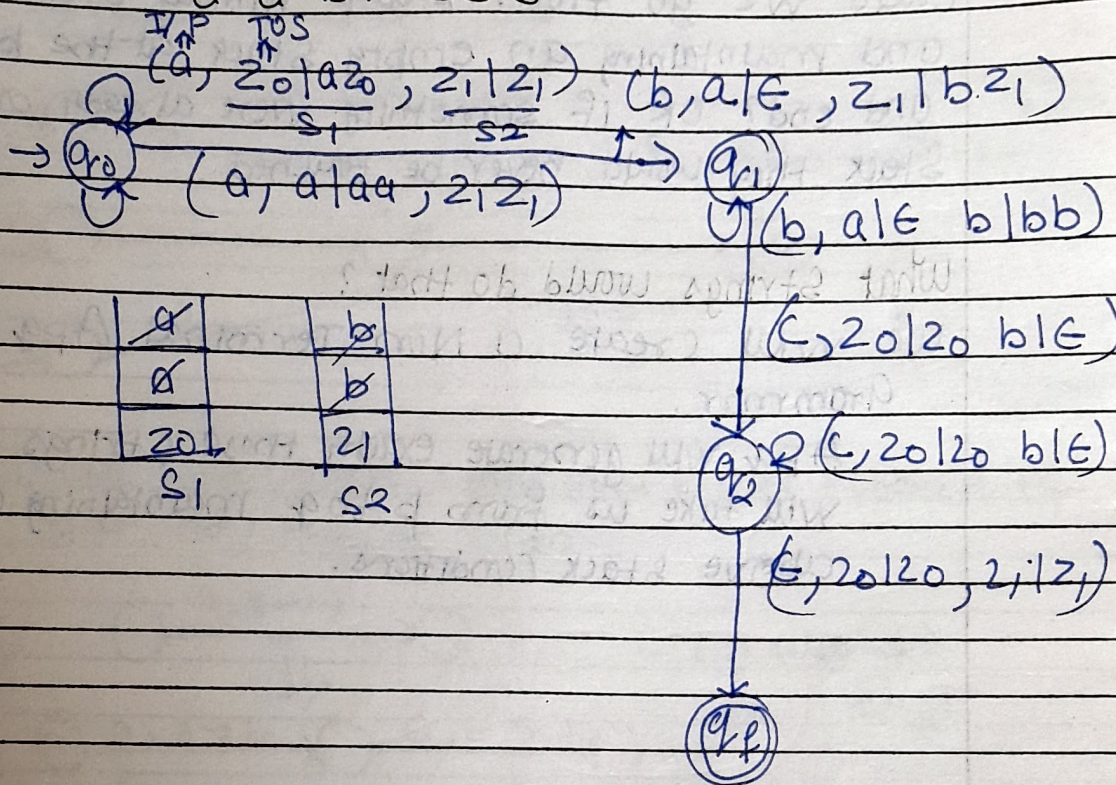
TWO Stack PDA.

$$\delta = Q \times E \cup \{ \epsilon \} \times \Pi \times \Pi \rightarrow Q \times \Pi^* \times \Pi^*$$

$\uparrow \quad \uparrow$
 stack 1 stack 2

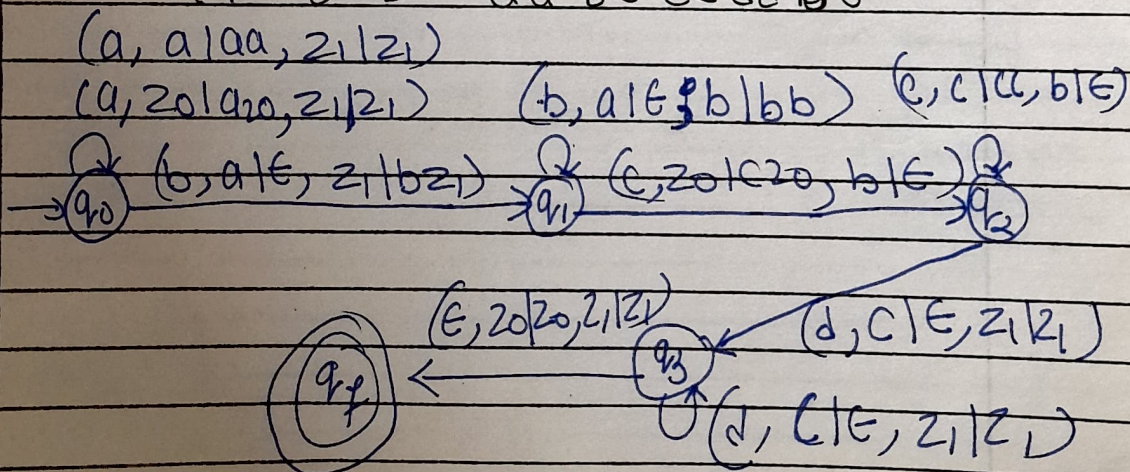
Q Ex:- $a^n b^n c^n$

$a a b b c c \epsilon$



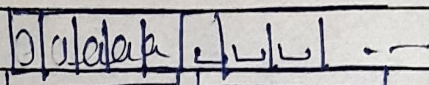
Q $L = a^n b^n c^n d^n$

Let $L = a a b b c c d d \epsilon$



Turing Machine (It's Deterministic)

Initial Configuration.

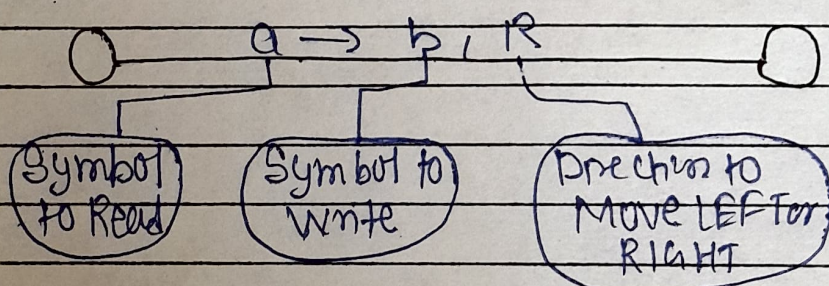


The I/p string Blanks out to Infinity

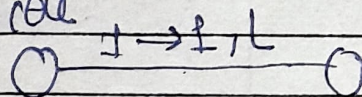
Operations on the Tape:

- Read / Scan Symbol below the Tape Head
- Update / Write a Symbol below the Tape Head
- Move the Tape Head one step LEFT
- Move the Tape Head one step RIGHT

ⓐ If we are at the left end of the tape and trying to move LEFT, then do not move. Stay at the left end.



If you don't want to update the cell
Just Write the Same Symbol



→ There are 2 final states

- a) Accept State
- b) Reject State

→ Computation can either

- 1) Halt and Accept
- 2) Halt and Reject
- 3) Loop (the m/c fails to halt)